

DESAFÍO 4: “EL ROBOT GOLEADOR”

Control de un robot G1 para localizar la pelota, aproximarse y ejecutar una patada

Historia del problema

El partido está por definirse. En la cancha virtual hay un robot humanoide G1 y una pelota. El robot no recibe una secuencia fija de movimientos: debe observar el entorno, entender dónde se encuentra, detectar dónde está la pelota, acercarse con estabilidad y ejecutar una patada.

Cada equipo deberá subir a Teams un archivo .py con su estrategia de control. Luego, los profesores cargarán ese archivo dentro del simulador oficial, ejecutarán la función de control repetidamente y registrarán si el robot logra completar la secuencia de manera correcta.

Objetivo general

Desarrollar una estrategia algorítmica para que el robot pueda:

- conocer su propia posición dentro de la escena;
- conocer la posición actual de la pelota;
- calcular la distancia y la dirección aproximada hacia la pelota;
- caminar hasta quedar en zona de pateo;
- estabilizarse antes del golpe;
- preparar la pierna y patear la pelota hacia el arco rival.

Importante

A diferencia de los desafíos de entrada y salida por consola, este ejercicio se evalúa por comportamiento en simulación. No alcanza con imprimir un resultado: el archivo entregado debe controlar al robot durante la ejecución.

Representación del entorno

Las posiciones del robot y de la pelota se expresan como coordenadas (x, y, z). Para decidir si el robot está cerca de la pelota, se recomienda trabajar con la distancia horizontal:

```
dx = posicion_pelota[0] - posicion_robot[0]
dy = posicion_pelota[1] - posicion_robot[1]
distancia = (dx**2 + dy**2) ** 0.5
```

La coordenada z representa altura. Puede ser útil para sensores y estabilidad, pero la aproximación a la pelota debe decidirse principalmente con x e y.

API disponible

El objeto robot entregado expone funciones de alto nivel. En el archivo de la competencia se invocan como métodos, por ejemplo: `robot.posicion_robot()`.

Función	Qué permite consultar o ejecutar	Uso dentro del desafío
<code>posicion_robot()</code>	Devuelve la posición actual del robot como (x, y, z).	Saber desde dónde parte y calcular distancia a la pelota.
<code>posicion_pelota()</code>	Devuelve la posición actual de la pelota como (x, y, z).	Determinar hacia dónde debe aproximarse el robot.
<code>velocidad_pelota()</code>	Devuelve la velocidad lineal de la pelota.	Detectar si la pelota está quieta o si ya fue pateada.
<code>estado_torso()</code>	Devuelve altura, pitch, roll y la bandera cayendo.	Evitar caminar o patear cuando el robot está inestable.
<code>pararse()</code>	Ordena una postura estable de pie con corrección de balance.	Inicializar, frenar, estabilizar y recuperar después de patear.
<code>caminar(velocidad=1.0)</code>	Hace caminar al robot con locomoción preentrenada.	Acercarse a la pelota mientras se recalcula la distancia.
<code>inclinarse(adelante=0.0, lateral=0.0)</code>	Inclina el torso hacia adelante o lateralmente.	Transferir peso antes de la patada.
<code>preparar_patada(pierna='derecha', fuerza=1.0)</code>	Carga la pierna elegida para preparar el golpe.	Separar la fase de preparación de la fase de impacto.
<code>patear(pierna='derecha', potencia=1.0)</code>	Ejecuta la extensión de la pierna.	Golpear la pelota cuando el robot está cerca y estable.
<code>mover_articulacion(nombre, angulo)</code>	Permite controlar una articulación puntual.	Uso avanzado; no es necesario para una solución válida.

Contrato de entrega

- La entrega será un único archivo con extensión **.py**.
- El archivo deberá definir obligatoriamente una función `control(robot)`.
- El servidor llamará a `control(robot)` muchas veces durante la simulación.
- La estrategia puede mantener estado mediante variables globales simples, por ejemplo `fase`, `contador` o `tiempo_fase`.

- El programa no debe usar `input()`, bucles infinitos internos, `time.sleep()` ni esperas bloqueantes.
- No se permite modificar el simulador, la escena ni `robot_api.py`.
- Solo se permiten librerías estándar de Python. No se permiten dependencias externas.

Tareas del desafío

Tarea 1: Inicializar y estabilizar

Al comenzar, el robot debe adoptar una postura estable usando `pararse()`. Si `estado_torso()` indica que el robot está cayendo, la estrategia debe priorizar la recuperación antes de caminar o patear.

Tarea 2: Detectar la posición del robot

En cada iteración se debe consultar `posicion_robot()` para saber desde dónde está actuando el robot. La solución no debe asumir una posición inicial fija.

Tarea 3: Detectar la posición de la pelota

La estrategia debe consultar `posicion_pelota()` de forma repetida. La pelota puede aparecer en posiciones distintas entre corridas, por lo que no se aceptan soluciones basadas en coordenadas hardcodeadas.

Tarea 4: Calcular distancia y decidir

Con la posición del robot y de la pelota se debe calcular la distancia horizontal. Si la pelota está lejos, el robot debe caminar; si está cerca, debe dejar de avanzar y pasar a una fase de estabilización.

Tarea 5: Aproximarse a la pelota

Mientras la distancia sea mayor que el umbral elegido, se deberá llamar a `caminar(velocidad)`. Se recomienda usar menor velocidad cuando el robot ya está cerca para evitar pasarse de largo o perder estabilidad.

Tarea 6: Preparar la patada

Antes de patear, el robot debe estar cerca de la pelota y no estar cayendo. Se puede usar `inclinarse()` de forma moderada y luego `preparar_patada()` para cargar la pierna elegida.

Tarea 7: Ejecutar la patada

La función patear() debe ejecutarse cuando el robot esté en zona de golpe. La estrategia debe evitar patear desde lejos o repetir la patada de forma caótica en todas las iteraciones.

Tarea 8: Recuperar postura

Luego de patear, el robot debe volver a una postura estable usando pararse(). Esta etapa permite finalizar la ejecución sin una caída inmediata posterior al golpe.

Estructura sugerida de solución

La forma recomendada de resolver el problema es mediante una máquina de estados. El siguiente fragmento no es una solución completa, sino un esquema de organización posible:

```
fase = 'inicio'
contador = 0
DISTANCIA_PATADA = 0.35

def control(robot):
    global fase, contador
    posicion_robot = robot.posicion_robot()
    posicion_pelota = robot.posicion_pelota()
    torso = robot.estado_torso()

    # 1. calcular distancia
    # 2. decidir fase
    # 3. ejecutar una acción de alto nivel
    # 4. actualizar contador o fase si corresponde
```

Casos mínimos que deberán contemplarse

- el robot comienza lejos de la pelota;
- el robot comienza relativamente cerca de la pelota;
- la pelota aparece en una posición distinta entre corridas;
- el robot se acerca demasiado rápido y debe estabilizarse;
- estado_torso() indica una posible caída;
- la pelota ya fue golpeada y comienza a moverse;
- la estrategia debe evitar patear antes de estar en zona de golpe;
- la estrategia debe terminar sin errores, aunque no logre convertir el gol.